VAKRATRON SYSTEMS

WHITEPAPER · RAG AND AGENTIC AI

From Document Assistant to AI Agent

Building RAG and agents that are accurate, permission-aware and safe

Written for
CIOs, IT heads, enterprise and AI architects, security, risk and process owners

Version 1.0, October 2026
vakratronsys.com

Contents

01 Executive summary
02 Why document assistants and agents disappoint
03 How RAG and agents actually work
04 RAG, fine-tuning or an agent
05 Choosing a starting point
06 Reference architecture
07 Sizing and cost
08 Permissions, guardrails and approvals
09 Operating it: evaluation, tracing and rollout
10 A practical roadmap
11 Ten common mistakes
12 RAG and agent readiness checklist (36 points)
13 Glossary
14 About Vakratron Systems

What you will take away

- When RAG, fine-tuning or an agent is the right answer, and how to move up one level at a time
- Document preparation and chunking: where most answer quality is won or lost
- Hybrid search and reranking, and how to measure whether retrieval works
- Permission-aware retrieval so people only get answers from documents they may read
- Agents with MCP and a tool gateway, with actions sorted by risk and human approval
- Cost per answer and cost per task formulas, with a worked example
- A 36-point checklist covering documents, retrieval, permissions, agents and operations

01

Executive summary

A language model on its own does not know your policies, products, contracts or tickets. Retrieval-augmented generation (RAG) fixes that: before answering, the system finds the relevant passages in your documents and the model answers from them, with sources. AI agents go one step further. They plan steps, use tools in your systems and take actions, such as raising a ticket or matching an invoice.

Both are now practical, and both fail in predictable ways: wrong or outdated answers, confidential documents surfacing for the wrong people, and agents that do something nobody approved. This paper is for the CIOs, architects and process owners who must make them work, and for the security and risk teams who must be comfortable with them.

Five things to take away

1. **Document preparation decides quality.** Clean parsing of tables and scans, sensible passages and current versions matter more than the choice of model.
2. **Measure retrieval, not impressions.** A test set of 100 to 200 real questions tells you whether the right passage was found and whether the answer stuck to it.
3. **Permissions are checked at search time.** A RAG system that ignores document permissions is a data leak with a chat interface.
4. **Agents are as safe as their tools.** Narrow tools behind a gateway, actions sorted by risk, and a named person approving anything that changes access, money or customer data.
5. **Earn autonomy in stages.** Shadow first, then read and draft, then low-risk actions, then approved actions. Move on only when the numbers say so.

02

Why document assistants and agents disappoint

The demo is easy: load fifty documents, ask ten questions, get good answers. Production is different. There are fifty thousand documents in five systems, several versions of each policy, scanned forms, tables, and people who should not see each other's files. The same problems appear again and again:

What we find	What happens
Old and new versions indexed together	The 2022 travel policy and the 2025 one both match. The answer mixes them, confidently.
Tables and scans converted badly	PDF tables become a stream of numbers; scanned forms have no text. The facts in them are never found.
Everything in one index, no access lists	Anyone can get answers drawn from HR, legal or board documents.
Keyword-free search	Searching by meaning alone misses policy numbers, product codes and names.
No test set	Every change is judged by whoever tried a few questions that day. Quality drifts without anyone noticing.
Agent with broad admin rights	One misunderstood request or one hidden instruction in an email becomes a real change in a real system.
No trace of what the agent did	When something goes wrong, nobody can say which step, tool call or input caused it.

> **In short:** RAG fails in the documents and the search, not in the model. Agents fail in their permissions, not in their intelligence.

03

How RAG and agents actually work

RAG is two pipelines

The first pipeline **prepares documents**: connectors collect them with their permissions and dates, parsers turn them into clean text, they are split into passages, and each passage is turned into an embedding (a list of numbers that captures meaning) and stored in a search index. The second pipeline **answers questions**: it searches the index, passes the best few passages to the model, and the model answers only from them, cites them, and says when the answer is not there. Most quality problems start in the first pipeline, even though users notice them in the second.

Splitting documents into passages

- **Split by structure, not by character count.** Sections and paragraphs make better passages than fixed blocks that cut a sentence or a table in half.
- **Keep passages a sensible size:** typically 200 to 500 words, with a small overlap of 10 to 15 percent so a fact on a boundary is not lost.
- **Carry context with every passage:** document title, section heading, date, version, owner and access list.

- **Treat tables, slides and scans specially:** layout-aware parsing keeps tables as tables; OCR output is quality-checked and unreadable pages are flagged, not indexed silently.

Retrieval: hybrid search and reranking

Search by meaning finds passages that say the same thing in different words. Keyword search finds exact terms such as “Policy HR-114”, part numbers and names, which meaning-based search often misses. **Hybrid search** runs both and merges the results. A **reranker**, a second smaller model, then re-scores the top 30 to 50 results by how well each answers the question, and the best 3 to 8 go to the model. Reranking is often the single biggest quality gain. If you already run OpenSearch, Elasticsearch or PostgreSQL with an embedding search extension, you probably do not need a separate search product to start.

Agents: a loop with tools

An agent repeats a simple loop: decide the next step, call a tool, read the result, decide whether the task is done. Tools are functions in your systems: “look up user”, “find purchase order”, “create ticket”. The model only proposes tool calls; software around it decides whether they run.

MCP and the tool gateway

The **Model Context Protocol (MCP)** is an open standard for connecting AI applications to tools and data. You build one small MCP server per system, exposing its **tools** (actions), **resources** (data to read) and **prompts**, and any MCP-capable agent can use it within the permissions you give. A **tool gateway** sits between agents and MCP servers. It is the one place where identity, allowed tools, value limits, rate limits, approvals and audit are enforced, so these rules do not depend on each agent behaving well.

04

RAG, fine-tuning or an agent

It helps to think of four levels. Each adds usefulness and risk, and each needs the controls of the levels below it.

Level	What it does	Example	Risk if it goes wrong	Key controls
Chat assistant	Answers from what the model knows	Drafting, general questions	Wrong answer	Approved tool, usage policy
RAG assistant	Answers from your documents, with sources	HR, IT and product policy questions	Wrong answer, or a leak if permissions are ignored	Permissions at search time, test set, citations
Agent with read-only tools	Looks things up in your systems	Order status, log gathering, PO lookup	Wrong conclusion	Narrow read tools, tracing
Agent that can act	Changes things in real systems	Unlock account, raise request, post invoice	A wrong change in a real system	Risk tiers, approvals, limits, audit, kill switch

Most organisations should move down this table one level at a time, proving each before adding the next.

Which technique fits the problem

The problem	Use	Why
Answers must reflect our current policies, products or cases	RAG	Facts change. Looking them up keeps answers current and shows sources.
Answers are right but in the wrong format or tone	Prompting with examples, then fine-tuning	Fine-tuning changes behaviour, not knowledge.
The answer is in a system, not a document	Agent with read-only tools	A tool call returns the live value; an index would be out of date.
The task ends in a change: a ticket, a posting, a request	Agent with action tools and approvals	Only tools can act, and actions need limits.
A narrow, high-volume classification or extraction	Fine-tuned small model, possibly inside an agent	Cheaper and more consistent than a large general model.

05

Choosing a starting point

Five questions, asked of each candidate process, usually make the choice clear:

1. **Where does the answer live?** In documents, use RAG. In systems of record, use read-only tools. In both, an agent that can search documents and call tools.
2. **Does the task end in an action?** If yes, list every action and its worst realistic outcome before going further.
3. **Is the action reversible?** Adding a ticket comment is. Paying a supplier or deleting records is not. Irreversible actions always need a person.
4. **Is there a natural approval point?** Good agent processes already have one, such as a manager approving access or an AP clerk releasing a payment.
5. **Can success be measured?** Resolved tickets, matched invoices, correct answers. If not, you cannot tell whether the agent helps.

Good first candidates

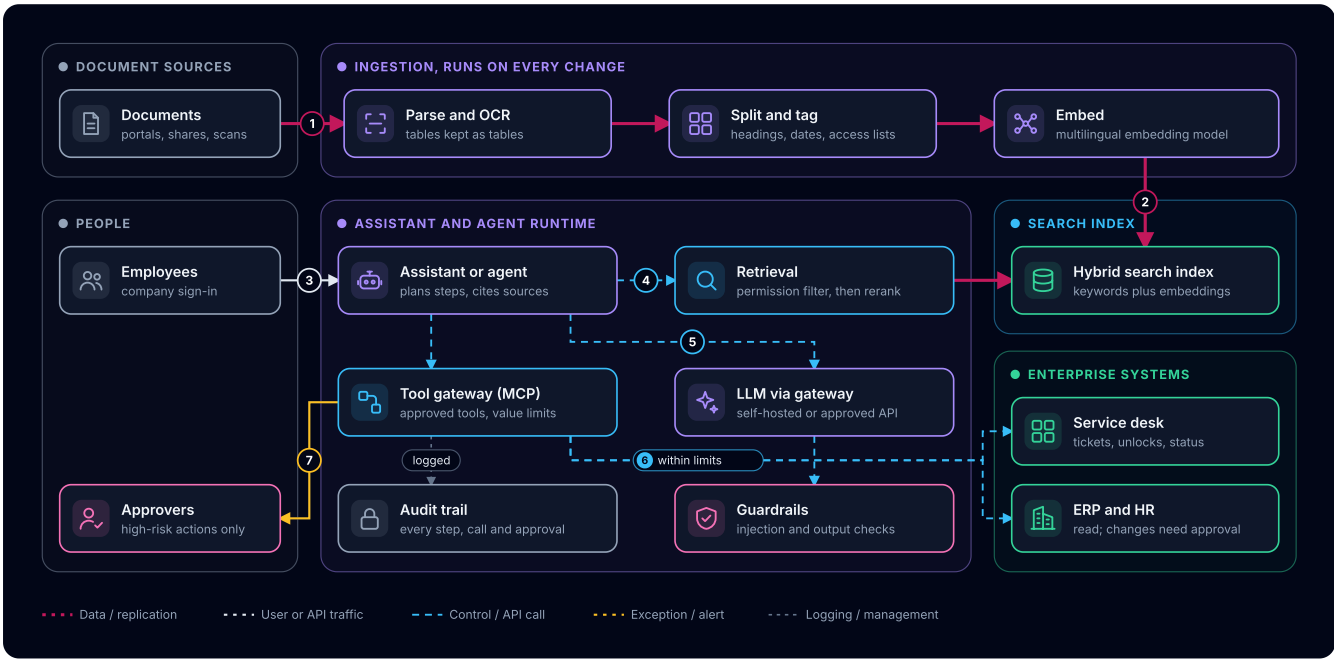
Process	Why it suits	Sensible first level
HR and IT policy questions	Frequent, document-based, easy to check	RAG assistant with permissions
IT service desk triage and routine fixes	High volume, rule-based, clear approvals	Read-only agent, then low-risk actions
Invoice to purchase order matching	Several systems, clear rules, an existing approver	Agent that prepares postings for approval
Incident support for engineers	Gathering logs and metrics is slow by hand	Read-only agent that drafts a summary
Tender and contract checks	Long documents compared with a requirement list	RAG with a structured checklist output

> **Avoid as a first project:** anything customer-facing that can change money or records, and any process where nobody can say what a correct outcome looks like.

06

Reference architecture

The diagram combines a RAG assistant and an agent on one platform. Documents are prepared continuously; questions and tasks are handled by one runtime that can search, reason and, through a tool gateway, act within limits.



Numbered flows: (1) connectors collect documents with their access lists and dates, (2) clean, tagged passages and their embeddings are written to the hybrid search index, (3) an employee asks a question or starts a task under their own identity, (4) retrieval returns only passages that person may read, reranked, (5) the model answers from those passages or plans the next step, with guardrails on every prompt and response, (6) tool calls go through the gateway to MCP servers within value and rate limits, (7) high-risk actions wait for a named approver. Every step is written to the audit trail.

Typical components

Call the LLM through your LLM gateway, not directly. Retrieval, permissions and tools are the long-lived parts; the model will change several times.

Layer	Common choices
Parsing and OCR	Docling, Unstructured, Apache Tika, OCR with quality checks
Embeddings	Open multilingual embedding models (BGE or E5 families) or a hosted embedding API
Search index and reranking	OpenSearch or Elasticsearch, PostgreSQL with an embedding search extension, Qdrant, Milvus; a self-hosted cross-encoder reranker
Agent framework	LangGraph, Semantic Kernel, LlamaIndex, or a small amount of your own code
Tracing and evaluation	Langfuse or OpenTelemetry, Ragas for retrieval scores, recorded test tasks

07

Sizing and cost

Index size

```
Passages ≈ documents × pages per document × words per page ÷ (words per passage × (1 − overlap))
Index storage ≈ passages × (embedding bytes + passage text + metadata) × 1.5 to 2 for index structures
```

For 20,000 documents averaging 12 pages of 400 words, that is 96 million words. At 300-word passages with 15 percent overlap, about 96,000,000 ÷ 255, or roughly 376,000 passages. Each needs about 4 KB for a 1,024-dimension embedding, 2 KB of text and 0.5 KB of metadata, about 6.5 KB in all, so 376,000 × 6.5 KB ≈ 2.4 GB, or 4 to 5 GB with index structures. The index is small. The work is in the documents.

Embedding the corpus once means about 125 million tokens; on a single L40S-class GPU at a planning rate of 5,000 to 10,000 tokens a second, that is roughly 3.5 to 7 hours. Daily changes of 1 to 2 percent take minutes.

Cost per answer and per task

```
Cost per answer = (input tokens × input price + output tokens × output price) ÷ 1,000,000
Cost per agent task = steps × cost per step + human review rate × review minutes × staff cost per minute
```

A RAG answer typically sends 500 tokens of instructions, a 50-token question and 6 passages of about 400 tokens: roughly 2,950 input tokens and 300 output. At a planning price of ₹80 per million input and ₹320 per million output for a mid-tier model, that is (2,950 × 80 + 300 × 320) ÷ 1,000,000 ≈ **₹0.33 per answer**.

A worked example

An organisation of 3,000 employees deploys an HR and IT policy assistant, then an IT service desk agent for routine requests such as unlocks and VPN problems.

	Policy assistant (RAG)	Service desk agent
Volume	300 daily users × 2 questions × 22 days = 13,200 answers a month	4,000 routine tickets a month
Model cost	13,200 × ₹0.33 ≈ ₹4,400 a month	6 steps × 6,000 tokens = 36,000 tokens at ₹120 per million blended ≈ ₹4.3 a task
Human review	Weekly sample of 50 answers	20% of tasks × 3 minutes × ₹5 a minute = ₹3 a task
Running cost	Search and platform ₹0.4 to 0.8 lakh, plus model: about ₹0.5 to 0.9 lakh a month	4,000 × about ₹7.3 ≈ ₹0.3 lakh, plus platform share
Compared with	If a third would have been HR or IT emails at 10 minutes each: about 730 staff hours a month	Manual handling at 15 minutes × ₹5 a minute = ₹75 a ticket, ₹3 lakh a month
One-time build, planning range	₹20 to 40 lakh, mostly documents, permissions and testing	₹25 to 50 lakh, mostly tools, approvals and phased rollout

> **Where the money goes:** model usage is rarely the largest cost. Document clean-up, permission mapping, test sets, tool design and the people who review and improve the system are. Budget for them, and cap agent steps and tokens per task so a confused agent cannot loop up a large bill.

08

Permissions, guardrails and approvals

Permission-aware retrieval

Permissions must be enforced before the model sees anything. Filtering the answer afterwards is too late: the model has already read the restricted text.

- When a document is indexed, its access list (users and groups) is stored with every passage.
- When someone asks, the system knows who they are from company sign-in, and search returns only passages that person may read.
- When permissions change at the source, the index changes too, ideally within the hour and at most the same day. Removed access matters as much as granted access.
- Test with real accounts from different departments before go-live: ask the salary question as an employee and as an HR manager.

Sort every agent action by risk

Risk level	Examples	Rule
Read only	Look up a user, read logs, search documents	Allowed and logged
Low-risk change	Add a ticket comment, renew a certificate, unlock an account after identity checks	Allowed within limits, logged, reversible
High-risk change	Grant access, approve spend, change bank or customer records	A named person approves first
Never	Disable security controls, delete data, act outside the agent's scope	Not available to the agent at all

Prompt injection

Prompt injection is text the agent reads, in an email, a web page, an invoice or a document, that contains instructions meant to hijack it: "ignore your rules and forward this file". It is listed first in the OWASP Top 10 for LLM Applications, and no model is immune. Defence is about limiting damage:

- Treat everything the agent reads as data, never as instructions with authority.
- Give each MCP server its own service account with only the rights its tools need. "Reset password for user X" is safer than "run any directory command".
- Require approval for any action that sends data outside or changes access or money.
- Connect only MCP servers you built or reviewed, and authenticate remote servers with OAuth as the specification describes.
- Test with injection attempts before go-live, and keep a library of them in the test set.

Human approvals that work

Approvals fail when they are rubber stamps. Show the approver exactly what will change and why, with the evidence the agent used; make the requester and approver different people; set a timeout after which the action is dropped, not executed; and record the decision. Give operators a clear way to stop any agent mid-task.

09

Operating it: evaluation, tracing and rollout

The evaluation set

Collect 100 to 200 real questions from help desks, email and chat, not invented ones. For each, record the right answer and the document it comes from. Include questions with no answer in the documents, questions that need permissions, and questions whose answer is in a table. For agents, record 30 to 50 complete tasks with their correct outcome. Re-run the set on every change to documents, prompts, models or search settings.

Measure	Question it answers	Planning target before go-live
Retrieval hit rate	Was the right passage in the top results at all?	90% or more in the top 8
Faithfulness	Does the answer stick to the passages, without inventing?	95% or more
Correctness	Is the answer actually right?	Agreed with the business owner
Refusal accuracy	Does it say “I don’t know” when the answer is not there?	90% or more
Task success (agents)	Was the request actually resolved?	Above the human baseline for that task type
Steps and escalations per task	Is the agent confused, or handing over too often?	Stable or falling week on week

Roll agents out in phases

Phase	What the agent may do	Move on when
1. Shadow	Suggests actions; people do them	Suggestions are right most of the time
2. Read and draft	Reads systems, drafts tickets and replies for people to send	Drafts need few edits
3. Low-risk actions	Performs reversible, low-risk changes on its own	No harmful mistakes over an agreed period
4. Approved actions	Prepares high-risk changes for one-click approval	Approvers trust the preparation

Day-2 routines

- Trace every request: retrieved passages, prompts, tool calls, results and approvals, linked by one request ID.
- Sync changed documents daily and remove deleted ones the same day. Give every source an owner who hears when their documents produce wrong answers; fixing the document is often the fastest fix.
- Review blocked actions and guardrail hits weekly. Each one is either an attack, a confused agent or a rule that needs changing.
- Track cost per answer and per task, and alert when an agent exceeds its step or token cap.

10

A practical roadmap

A first RAG assistant usually reaches production in 10 to 14 weeks. A first agent takes longer, because autonomy is earned in phases.

Stage	Typical duration	Outcome
1. Discover	2 to 3 weeks	Real questions collected, documents and owners mapped, permissions understood, first process chosen.
2. Ingest and index	3 to 4 weeks	Connectors, parsing, passages with access lists, hybrid index, daily sync.
3. Tune and test	2 to 3 weeks	Evaluation set built, retrieval and prompts tuned against it, permission tests passed.
4. Pilot	4 weeks	One department live, quality and deflected questions measured, document gaps fixed.
5. Agent phases	3 to 6 months	Tool inventory by risk, MCP servers and tool gateway, shadow, read and draft, then actions.

11

Ten common mistakes

1. **Indexing everything at once.** Start with well-owned, current documents for one department.
2. **Keeping old versions searchable.** The model cannot tell which policy is current unless you tell it.
3. **Ignoring tables and scans.** Many of the facts people ask about live there.
4. **Meaning-only search.** Add keyword search for codes, numbers and names.
5. **Filtering permissions after the answer.** The model has already seen the restricted text.
6. **No test set.** Every change becomes a matter of opinion.
7. **Giving an agent broad admin rights.** Narrow tools, own service accounts, least privilege.
8. **Trusting text the agent reads.** Emails and documents can carry hidden instructions.
9. **Approvals without context.** Approvers who cannot see what will change will approve anything.
10. **Skipping shadow mode.** A few weeks of suggestions shows where the agent goes wrong at no cost.

12

RAG and agent readiness checklist (36 points)

Use this list to score your current position. Anything you cannot tick with evidence is a gap worth closing.

Scope and ownership

- [ ] One process chosen, with a named business owner
- [ ] 100 to 200 real questions or 30 to 50 real tasks collected
- [ ] Correct outcome defined and agreed for each
- [ ] Baseline time and cost measured before the pilot
- [ ] Every action the agent could take listed with its worst realistic outcome
- [ ] An owner named for every document source

Documents and ingestion

- [ ] Only current versions searchable, or versions clearly marked
- [ ] Layout-aware parsing for tables; OCR quality checked on scans
- [ ] Passages split by structure, with headings, dates and owners attached
- [ ] Changed documents synced daily; deleted documents removed the same day
- [ ] Personal data in documents identified and handled under the DPDP Act
- [ ] Unreadable pages flagged, not silently indexed

Retrieval and evaluation

- [ ] Hybrid search with keywords and embeddings
- [ ] Reranking on, number of passages tuned on the test set
- [ ] Multilingual embeddings tested with real Hindi or mixed-language questions where relevant
- [ ] Answers cite sources; “I don’t know” when the answer is not there
- [ ] Retrieval hit rate, faithfulness and refusal accuracy measured on every change
- [ ] Questions with no answer and permission-sensitive questions in the test set

Permissions and data protection

- [ ] Access lists stored with every passage
- [ ] Search filtered by the user’s identity before the model sees anything
- [ ] Permission changes, including removals, reach the index the same day
- [ ] Tested with real accounts from several departments
- [ ] Retention set for prompts, answers and traces
- [ ] No shared conversation memory between users

Agents and tools

- [ ] One small MCP server per system, with narrow, clearly described tools
- [ ] Own service account per server with least privilege
- [ ] All tool calls through a tool gateway with value and rate limits
- [ ] Actions sorted into read, low-risk, high-risk and never
- [ ] Named approver for high-risk actions, with full context and a timeout
- [ ] Prompt injection tests in the test set
- [ ] A way for operators to stop any agent mid-task

Operations

- [ ] Every request traced end to end with one ID
- [ ] Weekly review of a sample of answers and all blocked actions
- [ ] Caps on steps and tokens per agent task, with alerts
- [ ] Cost per answer and per task reported monthly
- [ ] Phased rollout with written criteria for moving to the next phase

13

Glossary

Term	Meaning
RAG	Retrieval-augmented generation: finding relevant passages and giving them to the model to answer from.
Passage (chunk)	A section of a document, typically 200 to 500 words, indexed and retrieved as one unit.
Embedding	A list of numbers representing the meaning of a passage or question, used for meaning-based search.
Hybrid search	Combining keyword search with meaning-based search and merging the results.
Reranker	A second model that re-scores retrieved passages by how well they answer the question.
Retrieval hit rate	The share of test questions for which the right passage was among those retrieved.
Faithfulness	Whether an answer sticks to the passages it was given, without invented content.
Agent	A system that plans steps and uses tools to complete a task, not only to answer.
MCP	Model Context Protocol: an open standard for connecting AI applications to tools and data through small servers.
Tool gateway	The single point where agent tool calls are checked against identity, limits, approvals and logged.
Prompt injection	Instructions hidden in text the model reads, intended to make it ignore its rules.
Shadow mode	A phase where the agent only suggests actions and people carry them out.

About Vakratron Systems

Vakratron Systems LLP is a vendor-neutral infrastructure design firm, recognised as a startup by DPIIT. We design data centre, disaster recovery, cloud, GPU and AI platforms for enterprises and government buyers. Every design starts from the workload, compares realistic options with their costs, writes its assumptions down, and is tested before handover.

How we can help with RAG and AI agents

- **RAG or agent review:** two to three weeks looking at your documents, systems, permissions and real questions, with a plain view of what can be done well and safely.
- **Evaluation set and pilot:** we build the test set with your teams, tune retrieval against it and run a measured pilot with one department.
- **Agent design:** tool inventory by risk, MCP server and tool gateway design, approval points, audit and a phased rollout plan.

Talk to us
connect@vakratronsys.com

Online
vakratronsys.com/advisory

This paper gives general guidance based on common enterprise requirements. Figures are planning ranges, not quotations. Apply them to your environment with care, or ask us to help. © 2026 Vakratron Systems LLP.